Kevin Küster

Zusammenfassung: Kontextfreie Grammatik

Kontextfreie Grammatik

Nutzlose Symbole - Kontextfreie Grammatiken

$G = (V_N, V_T, P, S)$, $X \in V_N \cup V_T$, X nützlich, falls: $\exists w \in V_T^* : \exists \alpha, \beta \in (V_N \cup V_T)^*$ und $S \Rightarrow_G^* \alpha X \beta \Rightarrow_G^* w$

Algorithmus:

(i)
$V\text{-alt} := \emptyset$;
$V\text{-neu} := \{ A \in V_N \mid \exists w \in V_T^* : (A \to w) \in P \}$
while $(V\text{-alt} \neq V\text{-neu})$:
  $V\text{-alt} := V\text{-neu}$;
  $V\text{-neu} := V\text{-alt} \cup \{ A \mid \exists \alpha \in (V_T \cup V\text{-alt})^* (A \to \alpha) \in P \}$;
end
$V_N' := V\text{-neu}$;
$P' := \{ (A \to \alpha) \in P \mid A \in V_N', \alpha \in (V_N' \cup V_T)^* \}$;
$G' = (V_N', V_T, P', S)$

(ii)
$V\text{-alt} := \emptyset$; $V\text{-neu} := \{S\}$;
while $(V\text{-alt} \neq V\text{-neu})$:
  $V\text{-alt} := V\text{-neu}$
  $V\text{-neu} := V\text{-alt} \cup \{ X \mid \exists A \in V\text{-alt} : \exists \alpha, \beta (A \to \alpha X \beta) \in P' \}$
end;
$V_N'' := V\text{-neu} \cap V_N'$; $V_T'' := V\text{-neu} \cap V_T'$
$P'' := \{ (A \to \alpha) \in P' \mid A \in V_N'', \alpha \in (V_N'' \cup V_T'')^* \}$

Epsilon & Kettenproduktion

$G = (V_N, V_T, P, S)$ kontextfrei, Produktion $(A \to B)$, $A, B \in V_N$ Kettenproduktion, Produktion $(A \to \varepsilon)$, $A \in V_N$, ε-Produktion

ε-Produktionen beseitigen

$S \to aW \mid d$
$R \to Z \mid XV$
$U \to aX \mid \varepsilon$
$V \to \varepsilon$
$W \to XY$
$X \to UV \mid VR$
$Y \to \varepsilon \mid cY$
$Z \to bbX$

1.) Alle $A \in V_N$ aus denen sich ε direkt oder indirekt ableiten lässt herausschreiben
$M_\varepsilon = \{ U, V, Y, X, R, W \}$,
$X \to UV \to \varepsilon V \to \varepsilon\varepsilon = \varepsilon$
[...]

2.) Grammatik ohne ε-Produktionen übernehmen

$S \to aW \mid d$
$R \to Z \mid XV$
$U \to aX$
↳ V nutzlos
$W \to XY$
$X \to UV \mid VR$
$Y \to cY$
$Z \to bbX$

3.) Kompensierung fehlender Wörter
⇒ Abkürzungsregeln für Alle $A \in M_\varepsilon$ auf der rechten Seite
$X \to UV \mid VR \mid U(V) \mid (U)V \mid V(R) \mid R(V) \mid$
↳ $UV \mid VR \mid U \mid V \mid R$
$S \to aW \mid d \mid a(W)$
⋮
$S \to aW \mid d \mid a$
$R \to Z \mid X$
$U \to aX \mid a$
$W \to XY \mid X \mid Y$
$X \to U \mid R$
$Y \to cY \mid c$
$Z \to bbX \mid bb$

Kettenproduktionen beseitigen

$S \to aW \mid d \mid a$
$R \to Z \mid X$
$U \to aX \mid a$
$W \to XY \mid X \mid Y$
$X \to U \mid R$
$Y \to cY \mid c$
$Z \to bbX \mid bb$

1.) Paare mit Kettenregeln sammeln (Transitiver abschluss):
$M_K = \{ (R,Z), (R,X), (R,U), (R,R), (W,X), (W,U), (W,R), (W,Z), (W,Y), (X,U), (X,R), (X,Z), (X,X) \}$

2.) Kettenregel durch übernehmen der Produktionsregeln (ohne weitere Kettenregeln) kompensieren:
(R,Z): $R \to \underbrace{bbX \mid bb}_{Z} \mid [\dots]$

$S \to aW \mid d \mid a$
$R \to bbX \mid bb \mid aX \mid a$
$U \to aX \mid a$
$W \to XY \mid aX \mid a \mid bbX \mid bb \mid cY \mid c$
$X \to aX \mid a \mid bbX \mid aX \mid a$
$Y \to cY \mid c$
$Z \to bbX \mid bb$

⇒ R, U, Z nutzlos:
$S \to aW \mid d \mid a$
$W \to XY \mid aX \mid a \mid bbX \mid bb \mid cY \mid c$
$X \to aX \mid a \mid bbX \mid aX \mid a$
$Y \to cY \mid c$

Normalformen für kfG

Umformungen

(U1) $A \to \alpha\beta\gamma$ $(\beta \in V_N \setminus \{A\})$ ersetze:
$A \to \alpha\beta_1\gamma \mid \dots \mid \alpha\beta_2\gamma$, $B \to \beta_1 \mid \dots \mid \beta_n$

(U2) Vor.: $A \to A\alpha \mid \beta$, $\alpha \neq \varepsilon$, $\beta \notin \{A\}(V_N \cup V_T)^*$
Wähle B: $B \to \alpha \mid \alpha B$ und $A \to \beta \mid \beta B$

(U3) Durchführung (U1), falls $B \neq S$ nutzlos
$\Rightarrow$ B inkl. Produktionen mit B entfernen

(U4) Wähle neues $B \in V_N$ und ersetze Satz von Produktionen
$X \to \alpha\beta_1\gamma, \dots, X \to \alpha\beta_n\gamma$ durch
$X \to \alpha B\gamma$ und $B \to \beta_1 \mid \dots \mid \beta_n$

(U5) Wähle neues $B \in V_N$ und ersetze Satz von Produktionen
$X_1 \to \alpha_1\beta\gamma_1, \dots, X_n \to \alpha_n\beta\gamma_n$ durch
$X_1 \to \alpha_1 B\gamma_1, \dots, X_n \to \alpha_n B\gamma_n$, $B \to \beta$

(U6) Vor.: $A \to A\alpha \mid \dots \mid A\alpha_k \mid \beta_1 \mid \dots \mid \beta_\ell$ $(k \geq 1)$, $\alpha_i \neq \varepsilon$,
$i \in \{1, \dots, k\}$, $\beta_j \notin \{A\}(V_N \cup V_T)^*$, $j \in \{1, \dots, \ell\}$
Wähle neues $B \in V_N$, ersetze A-Produktionen:
$A \to \beta_1 \mid \dots \mid \beta_\ell \mid \beta_1 B \mid \dots \mid \beta_\ell B$, $B \to \alpha_1 \mid \dots \mid \alpha_k \mid \alpha_1 B \mid \dots \mid \alpha_k B$

(U7) $a \in V_T$, $C_a \in V_N$, $C_a \to a$ hinzufügen;
ersetze alle Vorkommen von A in $A \to \alpha$ mit $|\alpha| \geq 2$ durch C_a

(U8) $A \to B\beta_1 \mid \dots \mid B\beta_n$, $|\beta_i| \geq 2$,
$A \to BX$, $X \to \beta_1 \mid \dots \mid \beta_n$, $X \in V_N$

Chomsky-Normalform

G kfG, $\varepsilon \notin L(G)$ in CNF, falls für alle Produktionen gilt:

$A \to BC$, $A \to a$; $B, C \in V_N$, $a \in V_T$

$\Rightarrow$ Zu jeder kfG G $\varepsilon \notin L(G)$ gibt es G'
in CNF mit $L(G) = L(G')$

CNF: 1. Beseitigung ε-Produktionen
2. Beseitigen Kettenproduktionen
3. Separieren von Terminalen / Non-Terminalen
4. „Zerschlagen" der Produktionen $A \to X_1 \dots X_m$ $m > 2$

1.), 2.) siehe oben

Beispiel

$S \to aW \mid d \mid a$
$W \to XY \mid aX \mid a \mid bbX \mid bb \mid cY \mid c$
$X \to aX \mid a \mid bbX \mid aX \mid a$
$Y \to cY \mid c$

3.) Für jedes Terminal ein nicht-Terminal einführen

$S \to aW \mid d \mid A$
$W \to XY \mid AX \mid a \mid BBX \mid BB \mid CY \mid c$
$X \to AX \mid a \mid BBX \mid AX \mid a$
$Y \to CY \mid c$
$A \to a$
$B \to b$
$C \to c$

4.) Produktionen mit zu viel NT durch einführen
von weiten NT verkürzen:
$W \to BBX \Rightarrow W \to BX_1$, $X_1 \to BX$

Rechtslineare Normalform

Grammatik in rechtslinearer Normalform, falls:
Jede Produktion: $A \to \varepsilon$; $A \to a\beta$; $a \in V_T$; $A, \beta \in V_N$
$\Rightarrow$ Jede rechtslineare Grammatik lässt sich
in rechtslineare Normalform überführen

1. Einführung Nicht-Terminal E mit Produktion: $E \to \varepsilon$
2. Jede Produktion $A \to w$, $w \in V_T \Rightarrow A \to wE$
3. Kettenproduktionen entfernen
4. Verkürzen / „Zerschlagen"

Beispiel

$G = (\{S, X\}, \{a, b\}, P, S)$ P: $S \to aabS \mid bbaS \mid X$
$X \to ba \mid ab$

1.) & 2.) $S \to aabS \mid bbaS \mid X$
$X \to baE \mid abE$
$E \to \varepsilon$

3.) $S \to aabS \mid bbaS \mid abE \mid baE$
$E \to \varepsilon$

4.) $S \to aU \mid bV$ $\quad Y \to bS$
$U \to aY \mid bE$ $\quad Z \to aS$
$V \to bZ \mid aE$ $\quad E \to \varepsilon$

Greibach - Normalform

Grammatik in GNF, falls für alle Produktionen gilt:

$A \to a\alpha \quad , \quad a \in V_T \quad , \quad \alpha \in V_N^*$

$\Rightarrow$ Zu jeder kfG $G \in \notin L(G)$ gibt es G' in GNF mit $L(G) = L(G')$

Vorbereitung: Festlegung Reihenfolge $A_1, \dots, A_n \in V_N$

1.) (Zwischenschritt): Alle Produktionen haben Gestalt:

$A_i \to a\alpha$, $a \in V_T$, $\alpha \in V_N^*$ oder $A_i \to A_j\alpha$, $j > i$

oder $B \to A_j\alpha$ für neues $B \in V_N$

↳ (V1): Einsetzen rechter Seiten

↳ (VG): „Beseitigung von Linksrekursion"

```
for (i := 1 to n):
  for (j := 1 to i-1):
    ersetze alle Prod. A_i → A_j α durch
    A_i → β_1 α | ... | β_ℓ α,  A_j → β_1 | ... | β_ℓ   (V1)
  end;
  ggf. Beseitigung A_i → A_i α unter Einführung B (VG)
end;
```

$\Rightarrow$ Alle Produktionen der Gestalt:

$A_i \to a\alpha$, $a \in V_T$, $A_i \to A_j A_k \alpha$, $B_i \to A_j\alpha$, $\alpha \in V_N^*$

2.) GNF durch geeignetes Einsetzen rechter Seiten

```
for (i := n-1 downto 1):
  ersetze alle Prod. A_i → A_j α  ⇒  A_i → β_1 α | ... | β_ℓ α,
  A_j → β_1 | ... | β_ℓ   (V1)
end;
ersetze alle Prod. B_i → A_j α  ⇒  B_i → β_i α | ... | β_ℓ α,
A_j → β_1 | ... | β_ℓ   (V1)
```

Linksrekursion entfernen

$T \to Tb \mid a \quad \Rightarrow \quad ab^* \quad \Rightarrow \quad T \to a \mid aT' \,,\; T' \to bT' \mid b$

$T \to TbSb \mid TaS \mid baR \mid aba \mid cT \quad \Rightarrow \quad (baR + aba + cT)(bSb + aS)^*$

↳ Für Linksrekursive Teile eigenes NT einfügen:

$T \to baR \mid aba \mid cT \mid baRT' \mid abaT' \mid cTT'$

$T' \to bSbT' \mid aST' \mid bSb \mid aS$

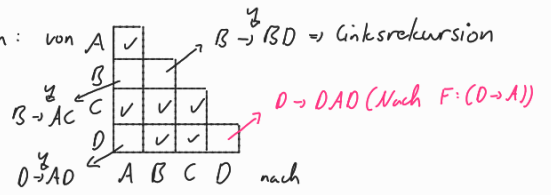
Kreise überbrücken

$A \to DA \mid a$
$B \to AC \mid BD \mid b$
$C \to DB \mid c$
$D \to AD \mid d$

Kreis: $A \to DA \to ADA$

$\Rightarrow$ Reihenfolge festlegen: A, B, C, D , $A_i \to A_j \dots \; i < j$

Verbotene Regeln:



Bei Fehler: Einsetzen rechter Seite von Ziel-Terminal:

F: (B → A):
$A \to DA \mid a$
$B \to \cancel{AC} \mid BD \mid b \mid DAC \mid aC$
$C \to DB \mid c$
$D \to AD \mid d$

F: (D → A):
$A \to DA \mid a$
$B \to b \mid DAC \mid aC \mid bB' \mid DACB' \mid aCB'$
$C \to DB \mid c$
$D \to \cancel{AD} \mid d \mid DAD \mid aD$
$B' \to DB \mid D$

F: (B → B):
$A \to DA \mid a$
$B \to \cancel{BD} \mid b \mid DAC \mid aC \mid bB' \mid DACB' \mid aCB'$
$C \to DB \mid c$
$D \to AD \mid d$
$B' \to DB' \mid D$

F: (D → D):
$A \to DA \mid a$
$B \to b \mid DAC \mid aC \mid bB' \mid DACB' \mid aCB'$
$C \to DB \mid c$
$D \to d \mid \cancel{DAD} \mid aD \mid dD' \mid aDD'$
$B' \to DB \mid D$
$D' \to ADD' \mid AD$

$A \to DA \mid a$
$B \to b \mid DAC \mid aC \mid bB' \mid DACB' \mid aCB'$
$C \to DB \mid c$
$D \to d \mid aD \mid dD' \mid aDD'$
$B' \to DB \mid D$
$D' \to ADD' \mid AD$

} Einsetzen rechter Seiten sin nicht-GNF :

$A \to \cancel{DA} \mid a \mid dA \mid aDA \mid dD'A \mid aDD'A$
$B \to b \mid DAC \mid aC \mid bB' \mid DACB' \mid aCB'$
$C \to DB \mid c$
$D \to d \mid aD \mid dD' \mid aDD'$
$B' \to DB \mid D$
$D' \to ADD' \mid AD$

$\Rightarrow$ Äquivalent für alle weiteren nicht passende NT

Pumping - Lemma kfS

$L \subseteq \Sigma^*$ kontextfrei:

$\exists n \in \mathbb{N} : \forall w \in L : |w| \geq n : w = u_1 v_1 \tilde{w} v_2 u_2$;
$u_1, v_1, u_2, v_2 \in \Sigma^*$:

1. $|v_1 v_2| \geq 1$
2. $|v_1 \tilde{w} v_2| \leq n$
3. $\forall i \in \mathbb{N} : u_1 v_1^i \tilde{w} v_2^i u_2 \in L$

Beispiel

$L := \{a^m b^m c^m \mid m \in \mathbb{N}\}$, $\Sigma := \{a, b, c\}$

Sei L kontextfrei: $n \in \mathbb{N}$, $w := a^n b^n c^n$
$\Rightarrow w \in L$, $|w| \geq 3n \geq n$

$\Rightarrow \exists u_1, v_1, v_2, u_2 \in \Sigma^* : w = u_1 v_1 \tilde{w} v_2 u_2$,
$|v_1 v_2| \geq 1$, $|v_1 \tilde{w} v_2| \leq n$, $\forall i \in \mathbb{N} : u_1 v_1^i \tilde{w} v_2^i u_2 \in L$

Also: $u_1 v_1^2 \tilde{w} v_2^2 u_2 \in L$ (*)

Da $w = a^n b^n c^n$ und $|v_1 \tilde{w} v_2| \leq n$ kann $v_1 \tilde{w} v_2$ nicht sowohl a als auch c enthalten:

$|v_1 v_2|_a = 0 \;\vee\; |v_1 v_2|_c = 0$ (**)

Wegen (*): $n + |v_1 v_2|_c = n + |v_1 v_2|_b = n + |v_1 v_2|_a$

$\Rightarrow |v_1 v_2|_a = |v_1 v_2|_b = |v_1 v_2|_c \overset{(**)}{=} 0$

↯: $|v_1 v_2| \geq 1 \Rightarrow L$ nicht kontextfrei

CYK - Algorithmus

→ Entscheidung, ob ein nichtleeres Wort in $L(G)$ liegt. ↳ G in CNF

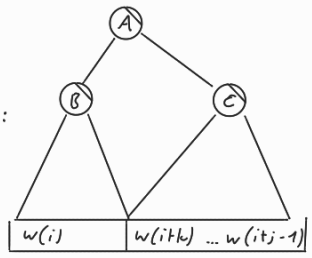
Schreibweise: $w[i,j]$ Teilzeichenreihe der Länge j ab Zeichen Nr. i

$V_{i,j} := \{ A \in V_N \mid A \Rightarrow^* w[i,j] \}$, $i \in \{1,\dots,n\}$, $j \in \{1,\dots,n+1-i\}$, $n := |w|$

↳ $w \in L(G) \Leftrightarrow S \in V_{1,n}$

↳ $V_{i,1} = \{ A \in V_N \mid (A \to w(i)) \in P \}$, $i \in \{1,\dots,n\}$

Ableitungsbaum:



Algorithmus:

```
n := |w|
for (i := 1 to n):
    V_{i,1} := { A ∈ V_N | (A → w(i)) ∈ P }
end;
for (j := 2 to n):
    for (i := 1 to n+1-j):
        V_{i,j} := ∅
        for (k := 1 to j-1):
            V_{i,j} := V_{i,j} ∪ BS(V_{i,k} V_{i+k,j-k})
        end;
    end;
end;
```

Beispiel

$G = (\{S, A, B, C\}, \{a, b\}, P, S)$ P: $S \to AB \mid BC$

$A \to BA \mid a$

$B \to CC \mid b$

$C \to AB \mid a$

$w = baaba$, Tabelle: (i,j) , $i \in \{1,\dots,n\}$, $j := \{1,\dots,n+1-i\}$

1.) Direkte Ableitung

w =	b	a	a	b	a
i =	1	2	3	4	5
j = 1	B	A,C	A,C	B	A,C

2.) Immer alle Kombinationen (1: BA, BC)

	1	2	3	4	5
1	B	A,C	A,C	B	A,C
2	S,A	B	S,C	A,S	

3.) ↓↗ : 1. BB, 2. AA, AC, SA, SC

	1	2	3	4	5
1	B 1.	A,C	A,C 2.	B	A,C
2	A,S 2.	B 1.	S,C	A,S	
3	∅	B	B		

4.)

	1	2	3	4	5
1	B	A,C	A,C	B	A,C
2	A,S	B	S,C	A,S	
3	∅	B	B		
4	∅	C,A,S			

5.) → BC, BA, BS, AB, SB, ...

	1	2	3	4	5
1	B 1.	A,C	A,C	B	A,C 4.
2	A,S 2.	B	S,C	A,S 3.	
3	∅ 3.	B	B 2.		
4	∅ 4.	C,A,S 1.			
5	A,S,C				

Da S vorkommt, lässt sich baaba aus S ableiten und liegt in der Sprache ✓